

HÁZI FELADAT

Programozás alapjai 2.

Terv

Barátfalvi Péter
WM53NO

2026. április 21.

TARTALOM

1.	Feladat.....	2
1.2	Feladat kibővítés.....	2
2.	Adattárolás és Fájlformátum.....	2
2.1	A központi katalógus.....	2
2.2	A kedvencek listája.....	3
3.	Felhasználói felület és Alapfunkciók.....	3
4.	Hibakezelés és Korlátok.....	3-4
5.	Terv.....	4
5.1	Objektum terv.....	4
	UML diagram.....	5
5.2	Metódusok.....	6-9
5.2	Algoritmusok.....	9
5.3	Tesztesetek.....	10

1. Feladat

A feladat neve: **Filmtár**

Készítsen filmeket nyilvántartó rendszert. Minden filmnek tároljuk a címét, lejátszási idejét és kiadási évét. A családi filmek esetében korhatár is van, a dokumentumfilmek esetében egy szöveges leírást is tárolunk. Tervezzon könnyen bővíthető objektummodellt a feladathoz!

Demonstrálja a működést külön modulként fordított tesztprogrammal! A megoldáshoz ne használjon STL tárolót!

1.2 Feladat kibővítés

A feladathoz hozzáadtam pár extra dolgot is. Amikor a felhasználó kilistázza a felvett filmeket, ilyenkor a jobb átláthatóság érdekében lehet rendezni. Például vetítési idő alapján csökkenő vagy növekvő sorrendbe állítani. Ezt szinten menüben lehet majd vezérelni. Lenne egy kereső funkció is ami a cím egy adott részét megtudná keresni és kilistázni a hasonló dolgokat. Továbbá másik plussz dolog a felhasználó kedvencek közé helyezhet olyan filmeket ami tetszik neki majd ezt akár egy txt fájlban exportálhatja is a könnyű nyomtatáshoz.

2. Adattárolás és Fájlformátum

2.1 A központi katalógus

A program az adatokat egy **filmtar.txt** fájlba tárolja, ide írja be az új adatokat és innen olvassa ki a meglévőket. Az adatok speciális formátumba vannak tárolva. Az adattagok pontosvesszővel vannak elválasztva, mivel a pontosvessző nem jelenik meg a filmnevekben ezáltal nem okoz beolvasási hibát. Minden tárolt film külön sorba kerül a fájlba, ez segíti az átláthatóságot. Minden egyes adatkupac (egy adott filmnek az adatai) egy speciális azonosítóval van ellátva. A dokumentumfilmek egy "D" betűvel vannak jelölve. A családi filmek pedig egy "CS" betűvel.

Példa a tárolt adatokra:

- CS;Shrek;90;2001;12
- D;Schumacher;112;2021;sport

2.2 A kedvencek listája

A program a **kedvencektar.txt** fájlba fogja tárolni a felhasználó által megjelölt filmeket. Itt nem tároljuk a filmek összes adatát csupán a nevüket és pontosvesszővel elválasztva a keletkezési évüket. Az összes adat külön sorba lesz írva. Betöltéskor a program ezek a nevek alapján keresi meg a filmek mutatóit.

Példa a tárolt adatokra:

- Shrek;2001
- Schumacher;2021
- King Kong;1933
- King Kong;2005

3. Felhasználói felület és Alapfunkciók

A program indításakor a filmtar.txt fájlból töltődnek be az adatok. Ezután a főmenü fogadja a felhasználót ami konzolos. Ha a felhasználó belépett egy almenübe azt megszakíthatja ha a "0" értéket adja meg vagy a megadott mégsem parancsot. Ezzel a felhasználó visszakerül a főmenübe és semmilyen mentés nem történik.

Ez a főmenü fogadja:

- 1. Film hozzáadása (speciális azonosító megadása, majd adatok bekérése)
- 2. Filmek kilistázása (opcionális módosítása, ezt a listát rendezni)
- 3. Film keresése(szövegdarabból keres címet)
- 4. Kedvenc filmek(ide gyűjti a kedvenceknek jelölt filmeket)
- 5. Film törlése
- 0. Kilépés és mentés

4. Hibakezelés és Korlátok

- **Menüválasztás:** Ha a felhasználó nem érvényes menüpontot, vagy szám helyett karaktert ad meg a program "Érvénytelen választás!" hibaüzenetet ad és újra kéri a választást.
- **Adatbevitel:** Ha a felhasználó az elvártnál különböző példáulul szám helyett karaktert vagy negatív számot ad meg akkor a program "Nem jó formátum" hibaüzenetet ad, majd megint adatbekérést végez.
- **Üres lista:** Ha a katalógus még üres és a felhasználó így szeretne módosítani vagy törölni hibaüzenetet kap, hogy "A katalógus még üres" ezután visszairányítja a főoldalra.

- **Dátumhiba:** Ha a felhasználó jövőbeli dátumot szeretne megadni a program hibát dob “Jövőbeli dátumot adott meg” és ismét bekérést kér.
- **Méretkorlát:** A programban nincs méretkorlát tehát tetszőleges nagyságú lehet a katalógus és az adott adatok hossza is. Mivel az adatok dinamikusan vannak tárolva ezért korlátlan a méretük csak a rendszermemória az egyetlen korlátja.
- **Nyelvkorlát:** A program csak az ékezet nélküli karaktereket tudja kezelni.

5. Terv

A feladat egy 6 osztályból álló, teljes UML objektummodell megtervezését valamint egy átfogó tesztprogram elkészítését igényli, amely kiemelt figyelmet fordít a beolvasási hibák és a kivételes esetek kezelésére.

5.1 Objektum terv

A UML objektummodell 6 osztályra van szétbontva, ezzel a felelősségek is elhatárolódnak egymástól.

Film: (Összostály) Ez az alaposztály összefogja az alap adattagokat továbbá a leszármazottak által majd módosított virtuális kiír() és virtuális mentes() funkciókat.

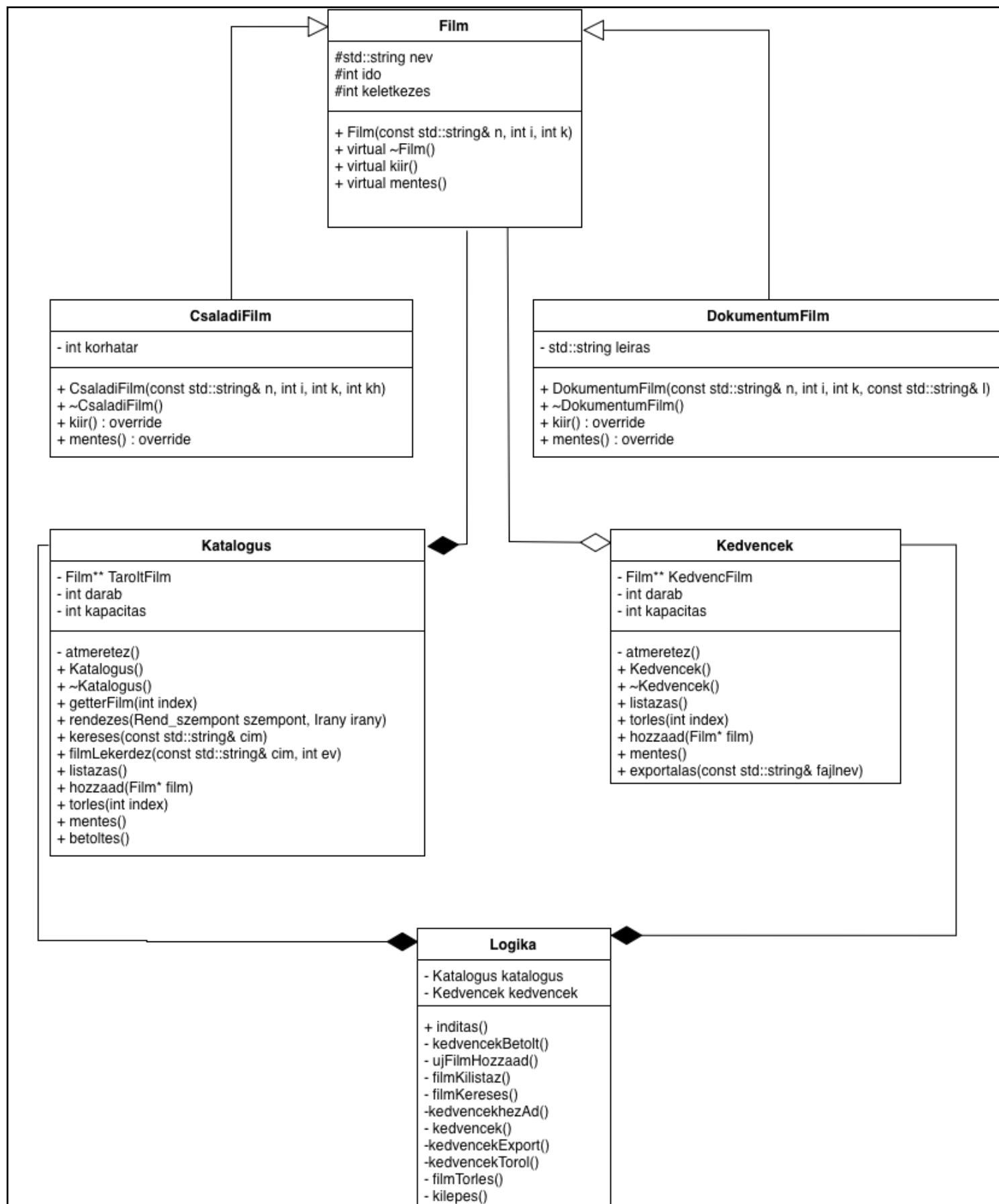
CsaládiFilm és DokumentumFilm: (Leszármazottak) Ez a két osztály egy plusz adattaggal van kibővíve a korhatárral/leírással és emiatt majd egyedi kiírás és mentés függvénye lesz.

Katalogus: ez a központi adattároló ez tárolja a filmeket. Ez a katalogus dinamikusan bővül, ezért szükség van a darabszámának és a kapacitásának is a tárolására. A legtöbb katalógus manipulátor függvény itt helyezkedik el. Például a rendezés, keresés, törlés, hozzáadás.

Kedvencek: ez a másodlagos adattároló csak a kedvencek jelölt filmeket tárolja. Ez az osztály manipulálja az adatait például a törlés és a hozzáadás függvényekkel.

Logika: ez az osztály kezeli az UI-t és adja ki a parancsokat a Katalogus/Kedvenc osztálynak. Feladata a menürendszer futtatása, a felhasználói

bemenetek kezelése, valamint az adatok bekérése. Továbbá birtokolja a Katalogus és Kedvencek osztályt.



5.2 Metódusok

Film (Ősosztály):

- **virtual kiir():** Ez egy virtuális kiíró metódus, ami kiírja az alap adatokat (nev, ido, keletkezés). Ezt fogják kiegészíteni a leszármazottjaik a saját adataikkal.
- **virtual mentes():** Ez egy virtuális mentés metódus, ami megfelelő formátumba menti el az adattagokat (nev, ido, keletkezés). Ezt fogják kiegészíteni a leszármazottjaik a saját adataikkal. Ezt a metódust a **Katalogus mentes()** funkció fogja hívni a program leállításakor, és ilyenkor kerül minden a katalógusban lévő adat kiírása fájlba.

CsaládiFilm (Leszármazott):

- **virtual kiir():** Ez a metódus felülírja az őosztály **virtuális kiir()** metódusát. Megjeleníti az alapadatokat (nev, ido, keletkezés), majd kiegészíti azokat a saját adatával (korhatar).
- **virtual mentes():** Ez a metódus felülírja az őosztály **virtuális mentés()** metódusát. A fájlba kiírást a “CS” azonosítóval kezdi majd, pontosvesszővel elválasztva az alap adattagok (nev, ido, keletkezés), majd a specifikus saját adatát (korhatar) a legvégén.

DokumentumFilm (Leszármazott):

- **virtual kiir():** Ez a metódus felülírja az őosztály **virtuális kiir()** metódusát. Megjeleníti az alapadatokat, majd kiegészíti azokat a saját adatával (leiras).
- **virtual mentes():** Ez a metódus felülírja az őosztály **virtuális mentés()** metódusát. A fájlba kiírást a “D” azonosítóval kezdi majd, pontosvesszővel elválasztva az alap adattagok (nev, ido, keletkezés), majd a specifikus saját adatát (leiras) a legvégén.

Katalogus

- **atmeretez():** Ez egy privát metódus. Ha a tárolt filmek száma eléri a kapacitás számát dinamikusan megnöveli azt, ezáltal több filmnek lesz hely. Amikor megtelik a tároló a meglévőnél, nagyobbbat foglal és a meglévő adatok mutatóit átmásolja azt az újba, a régi tömb memóriáját pedig felszabadítja.
- **getterFilm(int index):** Visszaadja a kapott indexű film mutatóját. Ez azért szükséges, hogy a **Logika** osztály átadhassa azt a **Kedvencek** osztály listájának.
- **listazas():** Sorszámozva 1-től kiírja a konzolra az összes tételt, amit a katalógus tárol. Egy ciklussal végighalad a tömb elemein, és minden elemre meghívja a **Film** osztály **virtuális kiir()** metódusát.
- **hozzaad(Film* film):** Ez a metódus felelős az új film beillesztéséért a katalógusba. Megkapja az új filmnek mutatóját, amit a **Logika** osztály hozott létre. Ha a kapacitás nem elég az új film eltárolásához, akkor dinamikusan bővíti azt az **atmeretez()** metódus hívásával. Ezután beteszi a mutatót a legutolsó helyre és növeli a darabszámot eggyel.
- **torles(int index):** Ez a metódus megkapja a törölni kívánt film indexét, és ezek után felszabadítja azt a memóriaterületet a delete operátorral, majd egy ciklussal az utána lévő mutatókat eggyel előre lépteti. Végezetül csökkenti a katalógus darabszámát.
- **mentes():** Ez a metódus egy ciklussal végiglépked a katalógus elemein, majd minden egyes elemre meghívja a **Film** osztály **virtuális mentes()** metódusát. Ezt a **filmtar.txt** fájlba menti ki.
- **betoltes():** Ez a metódus a program indításnál az adatok betöltéséért felel. A **filmtar.txt** fájlban lévő adatokat egy ciklus segítségével beolvassa. A sor elején lévő azonosító ("CS" vagy "D") alapján a new operátorral létrehozza a megfelelő típusú film objektumot, majd feltölti azt az adatokkal. Végül átadja a mutatót a **hozzaad(Film* film)** metódusnak.
- **filmLekerdez(std::string nev, int ev):** Ez a metódus egy film mutatóját adja vissza. Egy ciklussal végighalad a filmekben és ahol megegyezik a neve és a megjelenési éve annak visszaadja a mutatóját. Ha a metódus nem találja a keresett elemet **nullptr** értékkel tér vissza.

Kedvencek

- **atmeretez():** Ez egy privát metódus. Ha a tárolt kedvenc filmek száma eléri a kapacitás számát dinamikusan megnöveli azt, ezáltal több kedvenc filmnek lesz hely. Amikor megtelik a tároló a meglévőnél, nagyobb foglalt és a meglévő adatok mutatóit átmásolja azt az újba, a régi tömb memóriáját pedig felszabadítja.
- **listazas():** Sorszámozva 1-től kiírja a konzolra az összes tételt, amit a kedvencek tárol. Egy ciklussal végighalad a tömb elemein, és minden elemre meghívja a **Film** osztály **virtuális kiir()** metódusát.
- **torles(int index):** Ez a metódus megkapja a törölni kívánt kedvenc film indexét, és ezek után törli a tömbből az adott mutatót, majd egy ciklussal az utána lévő mutatókat eggyel előrébb lépteti. Végezetül csökkenti a kedvencek darabszámát. Csak a rámutató hivatkozást töröljük nem magát a film objektumot.
- **hozzaad(Film* film):** Ez a metódus felelős az új kedvenc film beillesztéséért a kedvencekbe. Megkapja a meglévő film mutatóját, amit a **Logika** osztály ad át. Ha a kapacitás nem elég az új kedvenc film eltárolásához, akkor dinamikusan bővíti azt az **atmeretez()** metódus hívásával. Ezután beteszi a mutatót a legutolsó helyre és növeli a darabszámot eggyel.
- **mentes():** Ez a metódus a kedvencek listáját menti el a kedvencektar.txt fájlba. Egy ciklussal végigmegy a tárolt mutatókon és minden film objektum **név** adattagját írja bele a fájlba.
- **exportalas(const std::string& fajlnev):** Ez a metódus paraméterként a kívánt fájlnevet kapja, majd ezen a néven létrehoz egy txt fájlt. Egy ciklus segítségével végiglépked a tároló mutatóin, és minden egyes mutatóra meghívja a **Film** osztály **virtuális kiir()** metódusát.

Logika

- **inditas():** Ez a program egyetlen publikus metódusa, ez inicializálja az adattagokat. Továbbá meghívja a betöltő függvényeket (**betolt()**, **kedvencekBetolt()**), és elindítja a főmenüt, amit a felhasználó vezérel.
- **kedvencekBetolt():** Beolvassa a kedvencektar.txt fájlból soronként az adatokat (**név, év**) majd a pontosvesszőnél szétbontja őket. Soronként meghívja a **filmLekerdez(const std::string& cim, int ev)** metódust mely visszatér az adott film mutatóját. Amennyiben érvényes mutatót kapunk vissza a **Kedvencek::hozzaad(Film* film)** metódusnak adja tovább a mutatót.

- **ujFilmHozzaad():** Ez a metódus hozza létre az új film objektumot. Bekéri a felhasználótól a film típusát, ezek után pedig a három fő adatot(nev, ido ,keletkezés), végezetül a speciális adattagot (**korhatar, leiras**). Létrehozza a megfelelő objektumot a new operátorral, majd átadja a **Katalogus::hozzaad(Film* film)** metódusnak.
- **filmKilistaz():** Ez a metódus meghívja a **listazas()** metódust, ami kiírja a konzolra a katalógusban tárolt tételeket.
- **filmKereses():** Ez a metódus bekéri a keresett címrészletet a felhasználótól, majd azt továbbadja a kereses() metódusnak. Végül a találatokat megjeleníti a konzolon.
- **kedvencek():** Ez a metódus felel ez egész kedvencek almenü irányításáért.
- **kedvencekExport():** Ez a metódus a kedvencek exportálásáért felel. Meghívja a **Kedvencek::exportalas(const std::string& fajlnev)** metódust ami elvégzi az exportálást.
- **kedvencekTorol():** Ez a metódus a kedvencek törléséért felel, bekéri a felhasználótól a törölni kívánt kedvenc indexét majd átadja azt a **Kedvencek::torles(int index)** metódusnak.
- **filmTorol():** Ez a metódus bekéri a felhasználótól a törölni kívánt film indexét majd meghívja rá a **Katalogus::torles(int index)** metódust.
- **kilepes():** Ez a metódus állítja le a programot és szabadítja fel a memóriát, továbbá elvégzi a kötelező mentéseket.

5.3 Algoritmusok

Rendezés: Ezzel a funkcióval a felhasználó rendezni tudja a katalógust. Három érték szerint lehet rendezni név, idő, keletkezési dátum. Továbbá egy gombnyomásra meglehet fordítani a sorrendet növekvőből csökkenőbe vagy fordítva. A kívánt rendezési mód például a 4-es, 5-ös, 6-os gombon lennének és a növekvő/csökkenő gomb egyben lenne mint egy kapcsoló a 7-es gombon. Maga a rendezést a Katalogus osztály csinálná egy buborék rendezéssel.

Keresés: Ez a funkció a 3. menüpont alatt lesz elérhető. Itt a felhasználótól lesz kérve egy filmcím-részlet, például “Harry” és erre az algoritmus egy for ciklussal végigmegy a katalógusban lévő összes filmen és ezen belül egy másik for ciklussal a megadott néven, hogy a rész hasonlít-e. Ha például a második betűnél megbukik az egyezés akkor továbblép az algoritmus a következő szóra. Ha nincs találat megy a következő névre.

5.4 Tesztesetek

Készítenék egy külön tesztmodult, amivel lehetne tesztelni a 4. pontban lévő hibaeseteket és korlátokat. Továbbá az egyik teszteset a dinamikus memóriakezelést ellenőrzi, hogy van-e szivárgás, és jól működnek-e az objektumok létrehozása, törlése. Ezzel megbizonyosodva róla, hogy a program kilépésekor nincs memóriaszivárgás.